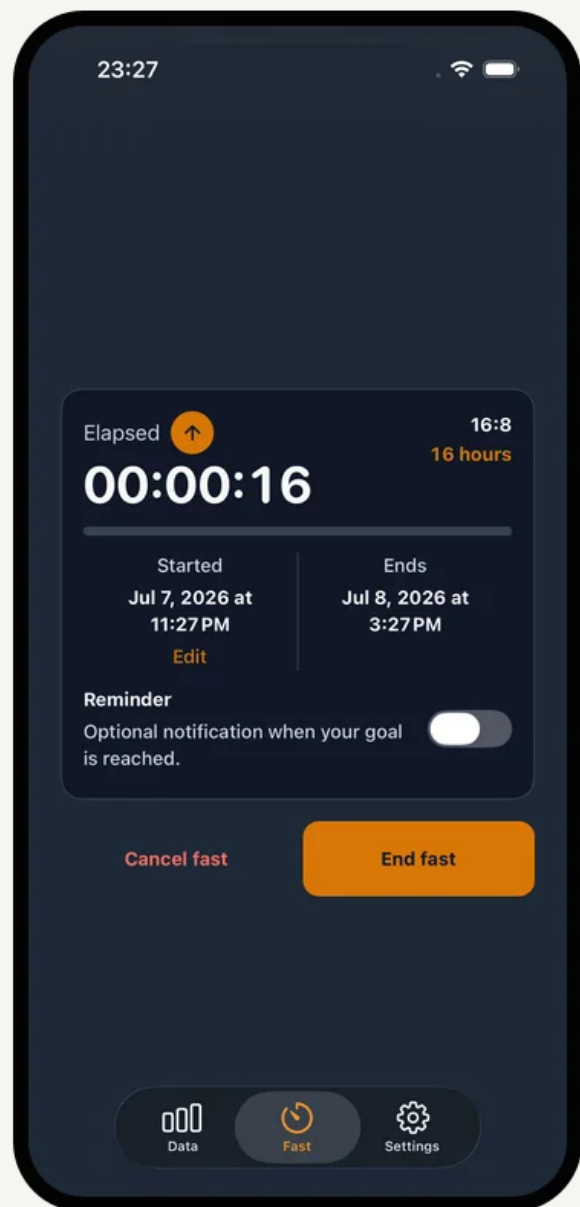
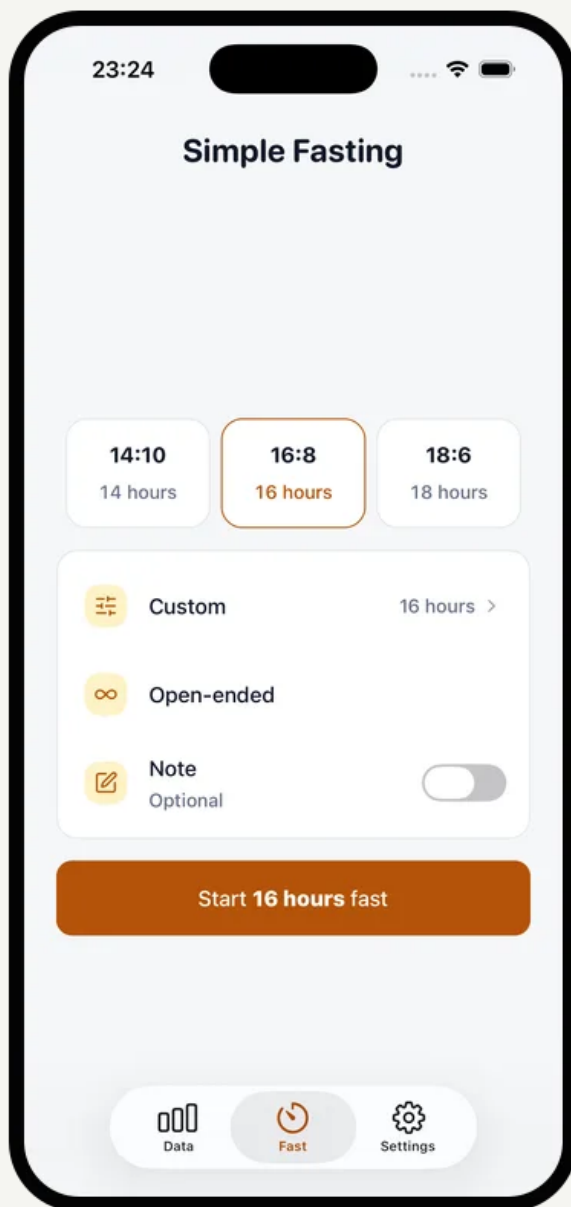


Case Study:

Simple Fasting

Designing a local-first mobile product



01

Product architecture

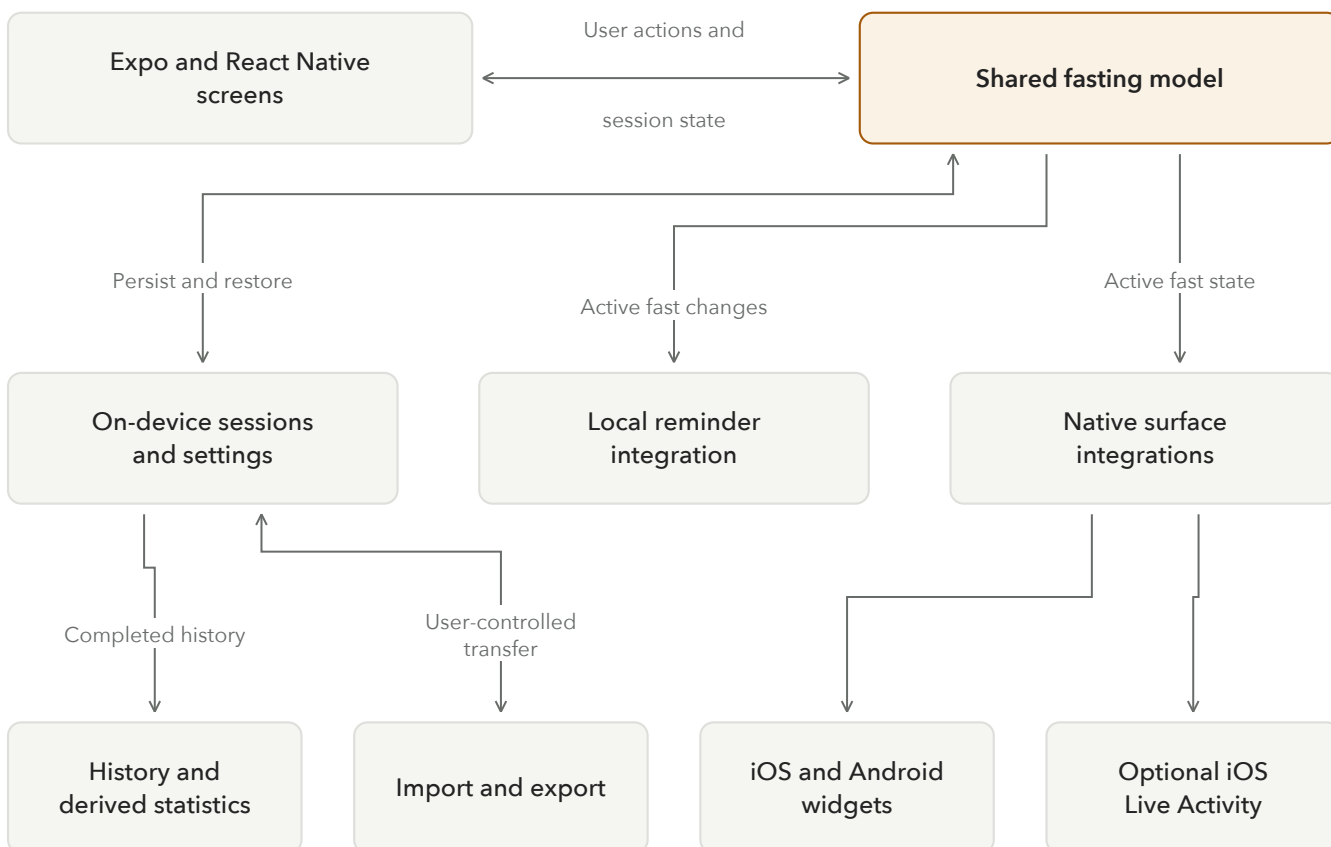
Simple Fasting is a cross-platform mobile app designed around an immediate daily interaction: open the app, start or stop a fast, and continue with the day. I built the product around on-device data and a shared fasting model so history, statistics, reminders and native widgets support that interaction without adding an account or a network dependency to it. The system design connects a small user experience to the state and platform boundaries needed to keep it coherent across iOS and Android.

Product goal

Make daily fasting easy to track, with personal data stored on the device. The timer stays central; goals, history, statistics and reminders support it. The architecture needs to keep those features in sync across screens and native surfaces while allowing each platform to use its own controls and behavior.

A shared product core with native integrations

Expo and React Native provide the shared app. The fasting model holds the meaning of the active fast, selected goal and completed sessions. Screens use that model, local storage preserves the data, and native integrations make the same active fast available outside the app.



Screens, reminders and widgets use the same fasting state. Statistics come from completed sessions in local history.

Shared code owns the product rules. Platform integrations handle notification scheduling, widgets and Live Activity. This keeps operating-system behavior out of individual screens and avoids giving each surface its own version of a fast.

02

Design system

The design system covers theme tokens, reusable components and screen layouts. It gives the timer, goals, history and settings a common structure while leaving platform behavior to native controls where possible.

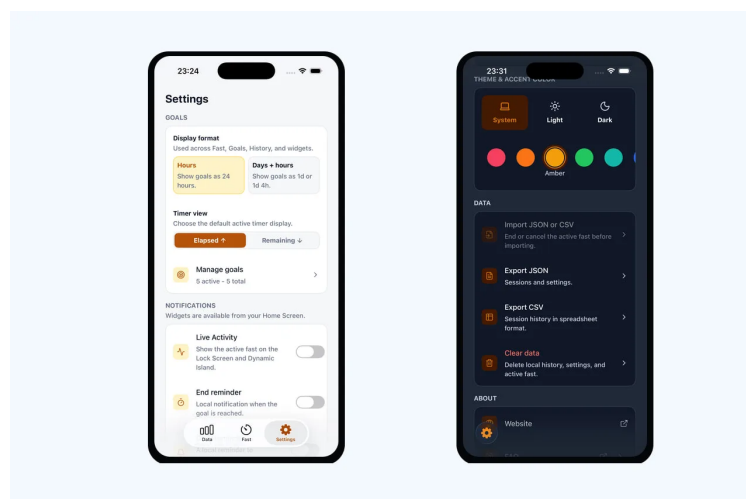
Layer	What is shared	What it controls
Theme tokens	Colors, spacing and text styles	Consistent appearance in light and dark modes
App components	Repeated sections, rows and action layouts	How common content is arranged across screens
Native controls	A common purpose with platform-specific presentation	Pickers, navigation and familiar interactions
Screen composition	Existing components connected to product state	What each screen shows and which actions it offers

The approach is to use native components exposed through Expo where they fit. Custom components cover app-specific content and layouts, while standard interactions keep their platform behavior. A date or time choice should feel familiar on iOS and Android without requiring both platforms to display an identical control.

The same principle applies to navigation and swipe gestures: use the platform's familiar behavior where available, and make the surrounding layout work with it. Safe areas, tab bars, keyboards and scrolling affect whether a control is usable, so they belong in screen design from the start.

Shared color tokens keep surfaces, text and controls consistent when the theme changes. Contrast and visibility are checked in both themes, including secondary text and the main action. Spacing and text styles come from the same system instead of being adjusted independently on every screen.

This structure makes changes easier to manage. A shared row or section can be improved once and reused. Platform differences stay close to the relevant control, while the screen code focuses on the fasting task.



Shared sections and theme rules keep settings consistent while controls retain their platform behavior.

State and local data

Consistent UI starts with shared state

Visual consistency is only one part of the design. The selected goal and active fast also need to agree across screens, navigation changes and native surfaces.

The important distinction is between temporary screen state and product state. Whether a picker is open belongs to the screen. The goal chosen through that picker belongs to the fasting flow. Moving to another tab should not reset that choice or create a different interpretation of the active session.

Screens read the shared fasting state and send user actions back to it. Widgets and Live Activity reflect that same session. History owns the completed records, and statistics use those records rather than maintaining separate totals.

This gives changes a clear path: update the session, then update the views and integrations that depend on it. A screen, widget or reminder should not contain an independent set of fasting rules.

Local data and user-controlled transfer

Fasting sessions, goals, notes and settings stay on the device. The core flow works without an account or server connection.

Information	Ownership and use
Active fast and selected goal	Shared application state used by the timer and native surfaces
Completed sessions and notes	Local history retained on the device
Statistics	Derived from local history
Reminder preferences	Local settings used by notification scheduling
Exported data	A copy the user chooses to take outside the app

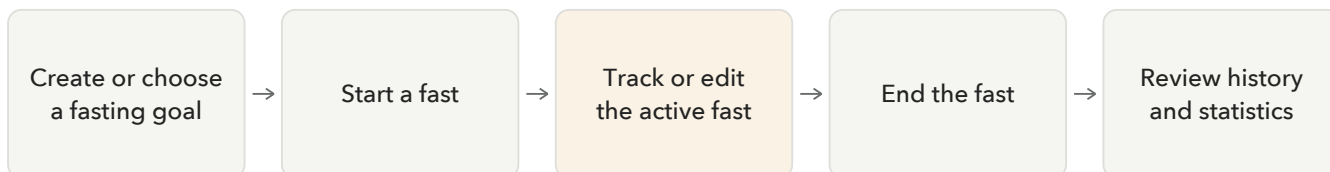
Import and export give the user a way to move their data. They work with the same session model used by the app, so imported history belongs in the same views and calculations as sessions recorded through the timer.

With no backend copy, storage and data transfer need careful handling. Keeping those concerns separate from screen code makes their rules easier to maintain as the interface changes.

04

One fast through the complete product

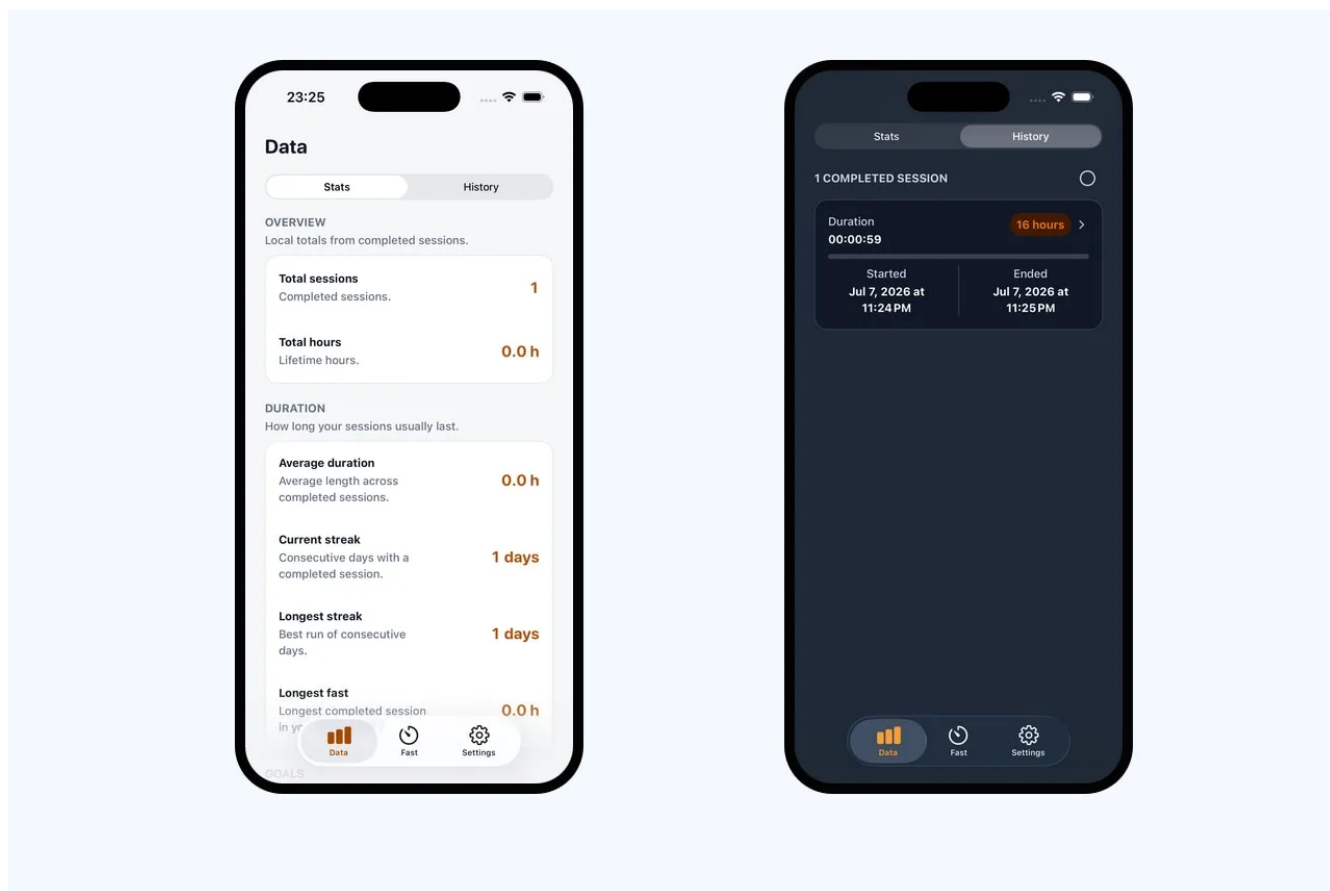
Users create reusable fasting goals in the app and choose one when starting a session. That goal remains part of the same flow through tracking and review.



One session connects goal selection, the active timer and the completed record.

Starting a fast establishes the state used by the timer, widgets and reminders. Editing it updates the related views and scheduled reminder. Ending it adds the completed session to history, where statistics can use it.

This flow also guides validation: the chosen goal survives navigation, native surfaces reflect the active fast, and the completed session appears in history. Testing the connected flow catches mismatches that isolated screen checks miss.



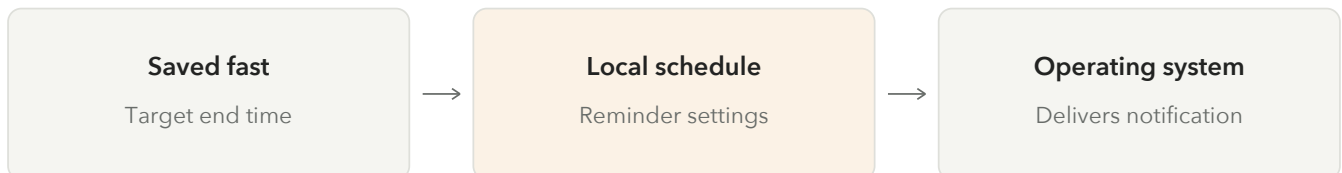
History and statistics read the same completed-session data.

05

Notifications and native surfaces

Schedule reminders from saved state

Notifications are scheduled locally from the saved fasting state and reminder settings. The end-of-fast reminder is tied to the session's target end time. The operating system delivers the scheduled notification; no server push is needed.



When the active fast changes, its reminder changes with it. Ending the session also removes the need for a pending end-of-fast reminder. A daily reminder is a separate schedule, so a recurring prompt and a session-specific notification have different purposes and settings.

Keep tracking independent of notifications

Notification permission does not control the timer. If permission is denied or reminders are turned off, starting, tracking and ending a fast still work. Scheduling belongs in the notification integration, rather than in a screen that may be closed or unmounted.

Use the same session on every surface

Widgets on iOS and Android show the same active fast as the app. The iOS Live Activity is optional and can be disabled without stopping the session. These surfaces display session state; they do not own a separate timer or history.

Check the native experience

Native behavior is checked on devices, while simulator and emulator checks cover layout, navigation and theme differences. This keeps both the shared logic and the platform-specific experience part of the same design.

Summary

Simple Fasting combines shared product state and UI components with native platform behavior. That structure supports new screens and integrations while keeping session data consistent and the daily tracking flow simple.